

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica



ELABORATO DI NETWORK AND CLOUD INFRASTRUCTURES

Prof. Giorgio Ventre

a.a. 2025-26

Studenti:

NICOLA IADANZA

LUCA DE FILIPPO

FRANCESCO MAINOLFI

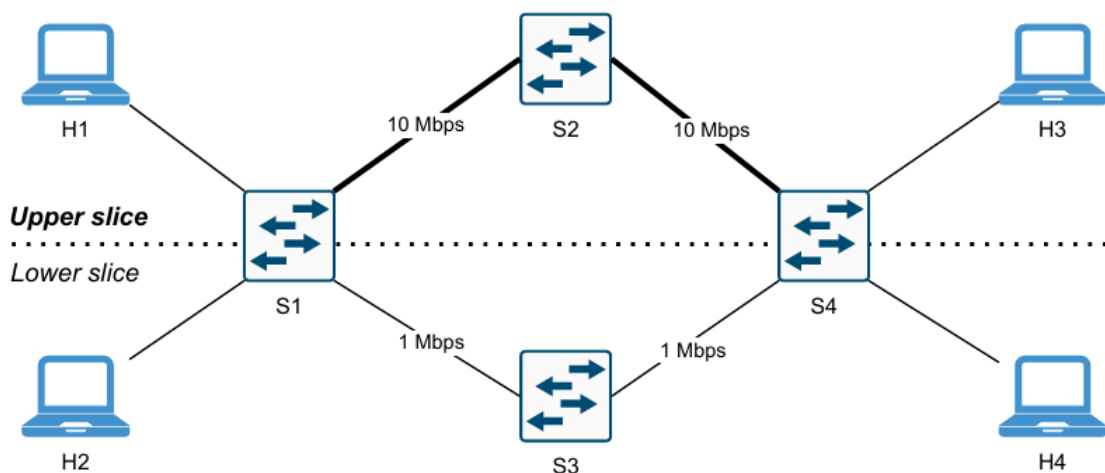
Obiettivi del progetto

Il progetto illustra lo sviluppo di un'architettura SDN basata sul **controller Ryu** e l'emulatore **Mininet** per implementare meccanismi avanzati di **Network Slicing**.

L'obiettivo primario è superare le rigidità delle reti tradizionali, garantendo isolamento tra tenant e una QoS rigorosa tramite il protocollo **OpenFlow**. Ispirandosi a modelli **DiffServ** per la classificazione dei pacchetti, l'infrastruttura è progettata per differenziare il traffico.

Di seguito è riportata la rappresentazione della topologia descritta nel file **topology.py**: questa presenta quattro Host (**H1, H2, H3, H4**), cinque Switch (**S1, S2, S3, S4, S5**) e **tre Slices diverse** per tre tipologie di traffico diverso:

- **Slice Video**: da **10 Mbps** dedicata al traffico UDP;
- **Slice Best-Effort**: limitata a **1 Mbps**, utilizzata di default dal traffico standard;



Fase 1: Installazione dei Prerequisiti

Aprire un terminale nella VM e installare tutto il software necessario eseguendo questi comandi:

```
sudo apt-get update
sudo apt-get install mininet python3-pip python3-flask
python3-requests -y
pip3 install ryu
```

Fase 2: Avvio dell'Infrastruttura

Aprire **tre terminali separati**, posizionarsi in tutti e tre nella cartella del progetto ed eseguirli in questo ordine:

1) Terminale 1: Il Controller

```
ryu-manager dynamic_slicing_controller.py ryu.app.ofctl_rest
```

Lasciare il terminale aperto e in vista. Qui compariranno i log fondamentali sulle decisioni dinamiche della rete.

2) Terminale 2: La Dashboard Web per il monitoraggio

```
python3 dashboard.py
```

Aprire un browser nella VM e andare su `http://127.0.0.1:5000`.

Sarà così possibile vedere i grafici del traffico (che inizialmente sono a zero).

3) Terminale 3: La Topologia

Prima di avviare, eseguire una pulizia preventiva con il comando `sudo mn -c`, poi lanciare la rete:

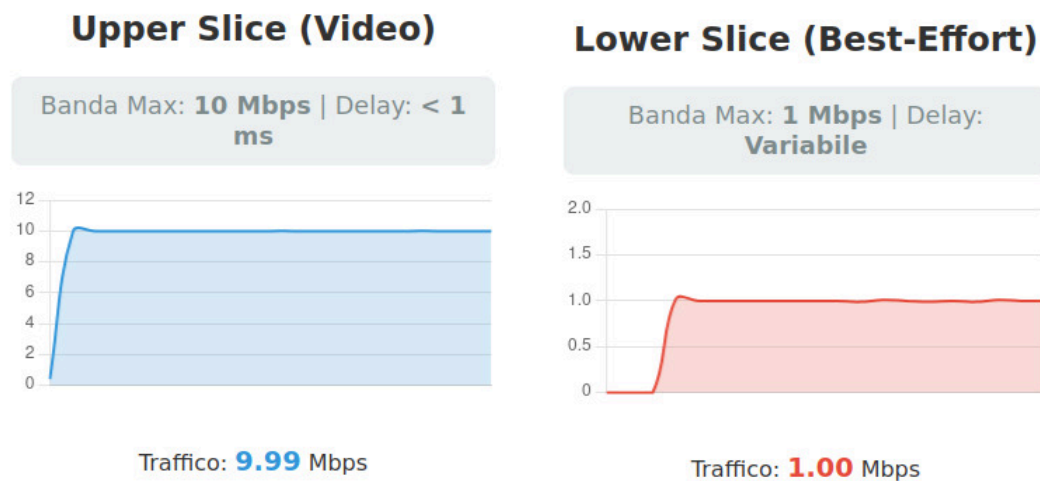
```
sudo python3 topology.py
```

Lo script farà **ADVANCED TRAFFIC GENERATION** di flusso video (max **10 Mbps**) e best-effort (max **1 Mbps**) tramite **ping automatico**.

Sarà possibile vedere il **66% di pacchetti dropped**, perché il controller SDN sta applicando il **Topology Slicing**, impedendo volutamente ai flussi isolati di incrociarsi.

```
Inizializzazione percorsi
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
```

Una volta che il **pingall** automatico è finito, è possibile accedere alla dashboard web è possibile visualizzare i grafici di **Upper Slice** e **Lower Slice** raggiungere i rispettivi valori di soglia.



Fase 3: Preparazione per i Test Dinamici

L'architettura prevede che il traffico normale (H2->H4) sia bloccato a 1 Mbps, ma se il flusso Video (H1->H3) è assente, il controller gli "presta" la slice veloce da 10 Mbps.

Nel **Terminale 3**, ripulite eventuali test automatici in background e aprite i server in ascolto:

```
sh killall iperf
h3 iperf -s -u -p 9999 &
h4 iperf -s -u &
```

Guardare il Terminale 1. Al comparire del log: **[DYNAMIC] Video scarico -> presto la slice VIDEO al traffico normale.**

Questo significa che il controller è pronto.

```
[DYNAMIC] Video scarico (0.47 Mbps) -> presto la slice VIDEO al traffico normale
(11625) accepted ('127.0.0.1', 56122)
127.0.0.1 - - [03/Sep/2026 10:52:24] "GET /stats/port/1 HTTP/1.1" 200 1803 0.001
177
```

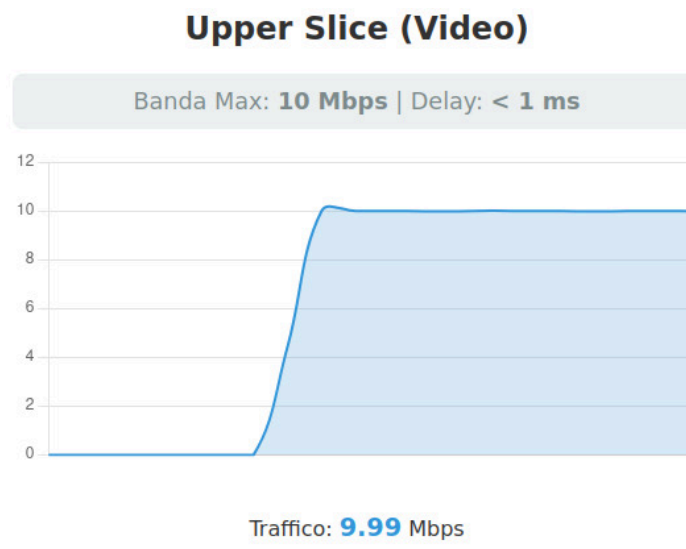
Fase 4: Esecuzione dei Test

Prova A: Rete Libera (Video Spento)

Verificare che il traffico normale riesca a rubare la banda non utilizzata dal video. Sempre nel **Terminale 3**, lanciare:

```
h2 iperf -c 10.0.0.4 -u -b 10M -t 15
```

Andando ad analizzare il risultato nella Dashboard, il grafico della slice video si alzerà a **10 Mbps** che corrisponde al **limite fisico del cavo**.



Al termine dei **15 secondi**, è possibile leggere nel **Terminale 3** la riga **Server Report**. La **Bandwidth** si attesterà a circa **9.75 Mbps**. Ciò ci dimostra che la prova è superata.

```
-----
Client connecting to 10.0.0.4, UDP port 5001
Sending 1470 byte datagrams, IPG target: 1121.52 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.2 port 48786 connected with 10.0.0.4 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.0177 sec 17.5 MBytes 9.76 Mbits/sec
[ 1] Sent 12463 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagram
[ 1] 0.0000-15.0754 sec 17.5 MBytes 9.72 Mbits/sec  4.601 ms 0/12462 (0%)
mininet>
```

Prova B: Rete Sotto Carico (Video Attivo)

Verificare che il controller protegga il traffico prioritario. Accendere il video per **60 secondi** e tentare di disturbare la rete.

Nel **Terminale 3**, avviare il video:

```
h1 iperf -c 10.0.0.3 -u -p 9999 -b 10M -t 20 &
```

Fermarsi e guardate il **Terminale 1**. Attendere finché non compare il log **[DYNAMIC] Video attivo -> il traffico normale torna sulla slice LENTA**.

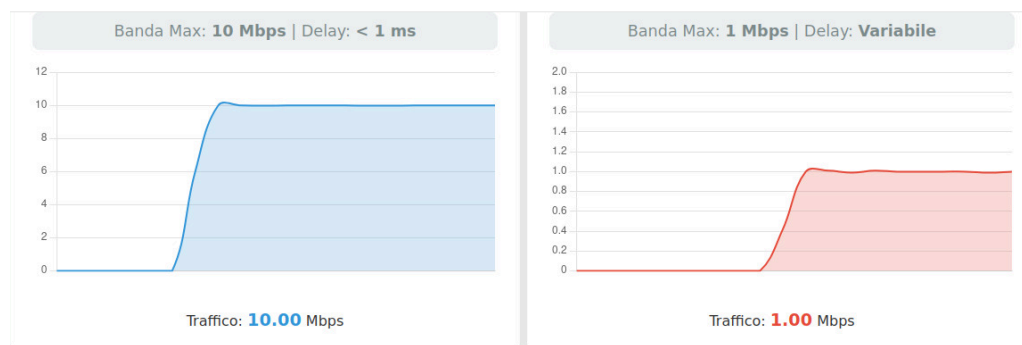
Questo è il **tempo di convergenza** necessario al controller per calcolare le medie e riprogrammare gli switch.

```
[DYNAMIC] Video attivo (4.92 Mbps) -> il traffico normale torna sulla slice LENTA  
(11625) accepted ('127.0.0.1', 38122)
```

Subito dopo aver visto il log, attivare il traffico normale:

```
h2 iperf -c 10.0.0.4 -u -b 10M -t 15
```

Andando ad analizzare il risultato nella **Dashboard**, il grafico di sinistra mostrerà il video in corso a **10 Mbps** per i restanti secondi, mentre il grafico di destra mostrerà il traffico normale "schiacciato" sul fondo.



Nel **Terminale 3** è possibile analizzare il **Server Report** di H2, la banda sarà crollata a circa **1.11 Mbps**. Ciò vuol dire che il controller ha ripristinato le priorità.

```
.....  
Client connecting to 10.0.0.4, UDP port 5001  
Sending 1470 byte datagrams, IPG target: 1121.52 us (kalman adjust)  
UDP buffer size: 208 KByte (default)  
.....  
[ 1] local 10.0.0.2 port 48218 connected with 10.0.0.4 port 5001  
[ ID] Interval      Transfer      Bandwidth  
[ 1] 0.0000-15.5089 sec 1.87 MBytes  1.01 Mbits/sec  
[ 1] Sent 1334 datagrams  
[ 1] Server Report:  
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams  
[ 1] 0.0000-16.0982 sec 1.87 MBytes  974 Kbits/sec  45.596 ms  0/1333 (0%)  
mininet>
```